

AI Operations (AIOps)

Module 3 Assignment — Infrastructure & Containerization

MUDDA MANIKANTA PRUTHVI RAJ, DA24B016 — September 19, 2026

1 Question 1: Single-Stage vs. Multi-Stage Docker

1.1 Naive and Multi-Stage Image Sizes

Both images build, run and serve the API identically (`/healthz` returned `{"status":"ok"}`; `/predict` returned `spam` for the prize message and `ham` for the meeting message).

Image	Disk usage	Content size
q1-api-naive	2.15 GB	534 MB
q1-api-multistage	762 MB	173 MB
Reduction	64.6%	67.6%

1.3 Why it is smaller.

The naive image is built on the full Debian-based `python:3.11` (which bundles `gcc`, `build-essential`, development headers and `git`) and, because of `COPY . .`, also keeps the training scripts `data_gen.py` and `train_model.py` and the generated `spam_dataset.csv`, none of which the running API needs. The multi-stage build performs the dataset generation and model training in the `builder` stage and copies only `/opt/venv`, `model.joblib` and `app.py` into a `python:3.11-slim` runtime stage, so the compilers, build tools and training files are discarded with the builder layers. This removes about 1.39 GB of disk usage (2.15 GB $\rightarrow$ 762 MB), mostly from the base-image swap, while the remaining 173 MB of content is essentially the `scikit-learn/pandas/FastAPI` virtualenv that both images must carry.

2 Question 2: Multi-Container Orchestration with Docker Compose

2.1 Redis Cache, Compose Stack and Evidence

2.2 Cache logic in the API.

In `app.py`, `POST /predict` builds the key `predict:<text>` (exact input text) and calls `GET` on Redis. On a **hit** it returns the stored label immediately, without touching the model. On a **miss** it runs `model.predict`, stores the label with `SETEX` (TTL = 300s), and returns it. The Redis client is created at startup from the `REDIS_HOST` and `REDIS_PORT` environment variables, and the response also carries a `cached` boolean (`{"label":"spam","cached":false}` on a miss, `true` on a hit) so hits are directly visible. The label was correct on both paths.

2.3 docker-compose.yml.

Two services are defined. `cache` uses the official `redis:7-alpine` image with a `redis-cli ping` healthcheck. `api` is built from `Dockerfile.multistage`, publishes port 8000:8000, and sets `REDIS_HOST=cache`, so the API reaches Redis through the service name as hostname on the default Compose network. `depends_on` with `condition: service_healthy` makes the API start only after Redis is ready. The startup log confirms the networking: `Connected to Redis at cache:6379`.

2.4 Cache-hit speedup.

The stack was started with `docker compose up --build`, and the same message was sent repeatedly with `curl -w "%{time_total}":`

Request	Response	Time
1st call (cache miss)	<code>"cached":false</code>	20.50 ms
2nd call (cache hit)	<code>"cached":true</code>	5.73 ms
Calls 3–10 (all hits)	<code>"cached":true</code>	1.26–3.53 ms (avg. 1.99 ms)

The first hit was about $3.6\times$ faster than the miss (72% lower latency), and the following hits averaged about $10\times$ faster (20.50 ms vs. 1.99 ms). The API log shows all ten `POST /predict` requests returning 200 OK.

2.5 Compose vs. Kubernetes.

A `Dockerfile` only describes how to build a single image and cannot express a multi-container application; `Compose` declares several services, their shared network, service-name DNS, environment and startup order in one file and runs them together with a single command. Its scope is a *single host*, whereas `Kubernetes` orchestrates containers across a *cluster of nodes*, adding scheduling, self-healing, scaling and rolling updates.

3 Question 3: Kubernetes Indexed Job: Parallel Data Validation

3.1 Parallelism Choice

The cluster has 2 nodes $\times$ 2 allocatable CPUs = 4 CPUs. Each pod parses only one 50-row shard, so half a core suffices. I chose `parallelism: 8` (equal to `completions`, so nothing higher is useful) with `CPU request = limit =`

500m: $8 \times 0.5 = 4.0$ CPUs, exactly the 4-CPU budget, with Guaranteed QoS; a 1-CPU request would cap the Job at 4 concurrent pods. System pods already request CPU (850m on `minikube`, 100m on `minikube-m02`), leaving 1150m and 1900m free, so $\lfloor 1150/500 \rfloor + \lfloor 1900/500 \rfloor = 2 + 3 = 5$ pods can run at once: 8 is the upper bound and 5 the expected effective concurrency. The Downward API exposes `metadata.name` and `spec.nodeName` to each pod.

3.2 Evidence of Achieved Parallelism

All 8 pods were created at once; their start times (`kubectl get pods -o wide, startTime`) show two waves:

Start (UTC)	Pods started	on minikube	on minikube-m02
13:43:09Z	5 (concurrent)	job-1-dwjj6, job-4-fxbk9	job-0-9hsc5, job-2-n544k, job-6-lxs7s
13:43:13Z	3 (were Pending)	job-3-sljzb, job-5-2hrjk	job-7-hxf2j

Five pods ran concurrently (2 on `minikube`, 3 on `minikube-m02`), exactly as predicted; the other 3 stayed `Pending` until CPU was freed. All 8 finished `Completed` with 0 restarts (4 per node).

3.3 Results via the Kubernetes API

`collect_results.py` lists the Job's pods through the Kubernetes API, reads each pod's log and parses its result line. A shared volume was avoided because `minikube`'s storage provisioner creates `hostPath`-backed volumes, i.e. a directory on a single node's disk; the pods ran on both nodes, so they cannot share one location, whereas API log reads work regardless of placement.

Shard	0	1	2	3	4	5	6	7	Total
Invalid rows (of 50)	4	5	6	7	8	9	10	11	60

All counts match the seeded ground truth (4 + shard index).

3.4 Parallelism with 3 Nodes $\times$ 2 CPUs

I would keep `parallelism: 8` and 500m, because `completions: 8` already caps concurrency. What changes is that 8 would now actually be reached: assuming each extra worker has the same ≈ 100 m system overhead as `minikube-m02`, the schedulable pods are $2 + 3 + 3 = 8$, with total requests still 4 of 6 CPUs. More CPU per pod would not help, since parsing 50 rows is not CPU-bound.

4 Question 4: Kubernetes Deployments: Self-Healing and Rolling Updates

4.1 Deployment and Service

Both manifests applied successfully. `kubectl get deployment` shows `READY 2/2`, `UP-TO-DATE 2`, `AVAILABLE 2`, the ClusterIP Service exposes port 80 (10.97.218.252), and the two pods were scheduled on different nodes.

4.2 Self-Healing

I deleted one replica with `kubectl delete pod` (suffix 4ktpv). `kubectl get pods -o wide` then showed a new pod (suffix 9wpfx, 1/1 `Running`, age 7s) next to the untouched `csbv8`, restoring 2 running replicas with no manual action. The responsible controller is the ReplicaSet controller: the Deployment owns the ReplicaSet `spam-api-685bf9bcbb`, which owns the pods. Its reconcile loop compares the live pods matching `app=spam-api` with the desired `replicas: 2`; after the deletion it saw $1 < 2$ and created a replacement from the pod template.

4.3 Rolling Update

I added a `version` field ("`v2`") to `/healthz`, built `spam-api:v2`, and updated the Deployment with `kubectl set image`. `kubectl rollout status` progressed one replica at a time ("1 out of 2 new replicas have been updated", then "1 old replicas are pending termination") and ended with "deployment spam-api successfully rolled out". `kubectl rollout history` lists revisions 1 and 2, the pods moved to a new ReplicaSet (685bf9bcbb $\rightarrow$ 75f84ff989), and `/healthz` returned `{"status": "ok", "version": "v2"}`. There is no downtime because the default RollingUpdate (`maxSurge 25%`, `maxUnavailable 25%`, which rounds down to 0 for 2 replicas) starts a new pod first and removes an old one only after the new pod passes its readinessProbe, so 2 ready pods stay behind the Service.

Had this been a Job, the server would never exit, so the Job would never complete. The Job's pod template is immutable, so `kubectl set image` would be rejected and shipping `v2` would mean deleting and recreating the Job, killing all pods at once (an outage), and `rollout status/history` do not apply to Jobs.

4.4 Deployment vs. Job

A Deployment maintains a *desired state indefinitely*: it keeps N ready replicas, replaces failed pods and swaps versions gradually, which suits a long-running, always-available API. A Job (Question 3) runs *finite work* until the required completions succeed and then stops without restarting finished pods, which suits batch work such as validating 8 shards where being "done" is the goal.